

Name: _____

Class: _____

Date: _____

Pseudocode Bootcamp

Reading & Writing AP Exam Pseudocode



The Big Picture

On the AP exam, code is written in **AP pseudocode** (text) or as blocks. It isn't Python or JavaScript, but it follows strict rules from the Exam Reference Sheet. Learn these rules and every code question gets easier. The levels get harder as you go!

Level 0 • Decode the Reference Sheet

Fill in the blanks to explain what each statement does.

AP pseudocode	What it does
<code>a ← expression</code>	Evaluates the expression, then _____ the result to variable a.
<code>DISPLAY(expression)</code>	_____ the value of the expression, followed by a space.
<code>INPUT()</code>	Accepts a value from the _____ and returns it.
<code>a MOD b</code>	The _____ when a is divided by b.
<code>RANDOM(a, b)</code>	A random integer from a to b, _____ both a and b.
<code>a = b a ≠ b</code>	Tests if values are _____ / not equal. (Does NOT assign!)
<code>NOT AND OR</code>	NOT flips true/false; AND needs _____ true; OR needs _____.
<code>IF (condition) { } ELSE { }</code>	Runs the first block if the condition is true; otherwise runs the _____ block.
<code>REPEAT n TIMES { }</code>	Runs the block exactly _____ times.
<code>REPEAT UNTIL (condition) { }</code>	Repeats the block until the condition becomes _____.
<code>aList ← [1, 2, 3]</code>	Creates a list. <code>aList[i]</code> is the element at index i; the first index is _____.
<code>FOR EACH item IN aList { }</code>	item takes the value of each element, in order from _____.
<code>APPEND(aList, value)</code>	Adds value to the _____ of the list.
<code>INSERT(aList, i, value)</code>	Puts value at index i; later elements shift _____.
<code>REMOVE(aList, i)</code>	Removes the element at index i; later elements shift _____.
<code>LENGTH(aList)</code>	Returns the _____ in the list.
<code>PROCEDURE name(p1, p2) { }</code>	Defines a procedure; p1 and p2 are its _____.
<code>RETURN (expression)</code>	Sends the value back to the caller and _____ the procedure right away.
<code>MOVE_FORWARD()</code> <code>ROTATE_LEFT()</code> <code>ROTATE_RIGHT()</code> <code>CAN_MOVE(dir)</code>	Robot moves one square, or turns _____. <code>CAN_MOVE</code> returns true if that square is _____.

Name: _____

Class: _____

Date: _____

Pseudocode Bootcamp

Level 1 · Translate & Trace

Python → AP Translator

Python	AP pseudocode
<code>x = 5</code>	_____
<code>x == 5</code>	_____
<code>x != 5</code>	_____
<code>x % 2</code>	_____
<code>print(x)</code>	_____
<code>nums[0] (first item)</code>	_____
<code>len(nums)</code>	_____
<code>nums.append(7)</code>	_____
<code>for n in nums:</code>	_____
<code>while x < 10:</code>	_____
<code>def area(w, h):</code>	_____
<code>return w * h</code>	_____

Watch Out!

- ← stores a value. = compares. Don't mix them up!
- AP lists start at index **1**, not 0.
- REPEAT UNTIL** stops when the condition is **true**, the opposite of a while loop. Flip the condition!
- RETURN** ends the procedure immediately.
- An index below 1 or above LENGTH causes an **error**.

1A · Assignment & MOD

```

x ← 7
y ← x MOD 3
x ← x + y
DISPLAY(x)
DISPLAY(y)

```

y = _____ x = _____
 Displayed: _____

1B · IF / ELSE

```

a ← 4
b ← 10
IF (a * 2 > b)
{ DISPLAY("big") }
ELSE
{ DISPLAY("small") }

```

a * 2 = _____ → condition is _____
 Displayed: _____

1C · Compound Conditions

```

temp ← 75
IF (temp > 80 OR temp < 50)
{ DISPLAY("stay in") }
ELSE
{ DISPLAY("go out") }
temp ← temp - 30
IF (temp < 50)
{ DISPLAY("jacket") }

```

Displayed: _____

1D · INPUT + IF/ELSE

```

n ← INPUT()
IF (n MOD 2 = 0)
{ DISPLAY(n / 2) }
ELSE
{ DISPLAY(n * 3 + 1) }

```

- The user types **6**. Displayed: _____
- The user types **7**. Displayed: _____
- In `n MOD 2 = 0`, the `=` is a _____, not an assignment.
- What input would display **1**? _____

Name: _____

Class: _____

Date: _____

Pseudocode Bootcamp

Level 2 · Loops & Level 3 · Lists

2A · REPEAT n TIMES

```
count ← 0
REPEAT 4 TIMES
{ count ← count + 3 }
DISPLAY(count)
```

Displayed: _____

2C · Infinite Loop?

```
k ← 10
REPEAT UNTIL (k = 0)
{ k ← k - 4 }
```

k goes 10, 6, 2, _____, ... This is an
_____ because k never equals 0.
Fix it: change the condition to _____

2B · REPEAT UNTIL Trace Table

```
n ← 1
REPEAT UNTIL (n > 30)
{ n ← n * 3 }
DISPLAY(n)
```

Pass	1	2	3	4
n	_____	_____	_____	_____

Loop runs _____ times. Displayed: _____

3A · List Procedures

```
nums ← [5, 2, 8]
APPEND(nums, 4)
INSERT(nums, 2, 9)
REMOVE(nums, 1)
DISPLAY(nums[2])
DISPLAY(LENGTH(nums))
```

After APPEND: _____

After INSERT: _____

After REMOVE: _____

Displayed: _____

3B · FOR EACH Trace

```
vals ← [3, 6, 1, 8]
total ← 0
FOR EACH v IN vals
{
  IF (v MOD 2 = 0)
  { total ← total + v }
}
DISPLAY(total)
```

v	3	6	1	8
total	_____	_____	_____	_____

In words, this code _____



Think:

When a list changes, rewrite the whole list after every line. INSERT shifts items right; REMOVE shifts them left.

Pseudocode Bootcamp

Level 4 · Procedures & Robots

4A · Mystery Procedure

```
PROCEDURE mystery(a, b)
{
  IF (a > b)
  { RETURN (a - b) }
  RETURN (a + b)
}
DISPLAY(mystery(9, 4))
DISPLAY(mystery(2, 6))
x ← mystery(mystery(3, 1), 5)
DISPLAY(x)
```

1. `mystery(9, 4)` = _____
2. `mystery(2, 6)` = _____
3. Inner call `mystery(3, 1)` = _____, so `x` = `mystery(2, 5)` = _____
4. Name the parameters: _____. Name the arguments in line 7: _____
5. Why doesn't the second RETURN run when `a > b`? _____

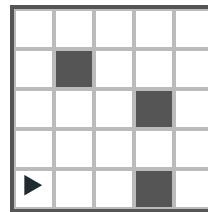
4B · Copies, Not Links

```
PROCEDURE addBonus(score)
{
  score ← score + 10
  RETURN (score)
}
s ← 80
t ← addBonus(s)
DISPLAY(s)
DISPLAY(t)
```

Displayed: _____

Why didn't `s` change? The parameter gets a _____ of the value.

4C · Robot Run



► = robot facing right
black = wall
The robot can't move into a wall or off the grid.

```
REPEAT UNTIL (NOT CAN_MOVE("forward"))
{ MOVE_FORWARD() }
ROTATE_LEFT()
MOVE_FORWARD()
```

1. How many times does the loop run?

2. Why does it stop?

3. Draw the robot's final square and an arrow for its direction. It faces _____.

4D · Procedures + Lists

```
PROCEDURE countBig(list, limit)
{
  count ← 0
  FOR EACH item IN list
  {
    IF (item > limit)
    { count ← count + 1 }
  }
  RETURN (count)
}
DISPLAY(countBig([4, 12, 7, 15, 9], 8))
```

1. Which items are greater than 8? _____
2. Displayed: _____
3. `countBig([1, 2], 5)` returns _____
4. What would go wrong if `count ← 0` were moved inside the loop? _____

Name:

Class:

Date:

Pseudocode Bootcamp

Boss Level · Write It Yourself

5A · Write a Procedure

Write a procedure `isEven(n)` that returns **true** if `n` is even and **false** otherwise.

5B · Write a Loop

`scores` is a list of numbers. Write code that displays every score that is **90 or higher**.

5C · Squash the Bug

This code should display the average of ANY list `scores`, but it's only right sometimes.

```
sum ← 0
FOR EACH s IN scores
{ sum ← sum + s }
avg ← sum / 3
DISPLAY(avg)
```

What's the bug? When is it still correct? Fix it.

5D · Exam Practice

★★ On Level

1. What is displayed?

```
x ← 2
REPEAT 3 TIMES
{ x ← x * x }
DISPLAY(x)
```

- A. 8
- B. 16
- C. 64
- D. 256

★★★ Exam Hard

2. Which code segment displays **1 2 3 4 5**?

A.

```
i ← 1
REPEAT UNTIL (i = 5)
{ DISPLAY(i) i ← i + 1 }
```

B.

```
i ← 0
REPEAT 5 TIMES
{ DISPLAY(i) i ← i + 1 }
```

C.

```
i ← 1
REPEAT UNTIL (i > 5)
{ DISPLAY(i) i ← i + 1 }
```

D.

```
i ← 1
REPEAT UNTIL (i > 5)
{ i ← i + 1 DISPLAY(i) }
```